MALWARE ANALYSIS LAB | REVIEW 2: IMPLEMENTATION

NPMShield: Static and Dynamic Analysis of Malicious npm Packages

Siri S Gowda | 23BCI0010
B.Tech CSE (Information Security), VIT Vellore
Faculty: Sanmuga Priya M
October 2026

18
real packages analysed

9 of 13
malicious ones flagged

0 of 5
benign false positives

Finalized methodology

1. Unpack archives on the VM, install nothing
2. Static analyzer scores code and package.json
3. Sandbox runs install scripts, then imports the package
4. Run twice: laptop profile and CI profile
5. Fuse: 0.4 static + 0.6 dynamic, canary floor of 80
6. Gap rule lists packages only the sandbox flags

Score bands: 0 to 30 low, 31 to 60 suspicious, above 60 malicious. Flagged means a score above 30.

Changes since Review 1

- Stand-ins for curl, wget and missing modules, so the package still shows its intent
- Fallback that imports every top-level file when main is missing
- Canary rule: an opened fake credential file forces a fused score of at least 80
- Gap rule for sandbox-flagged, static-clean packages
- Indicators split into attacker-controlled, legitimate services and low-confidence strings

System architecture and workflow

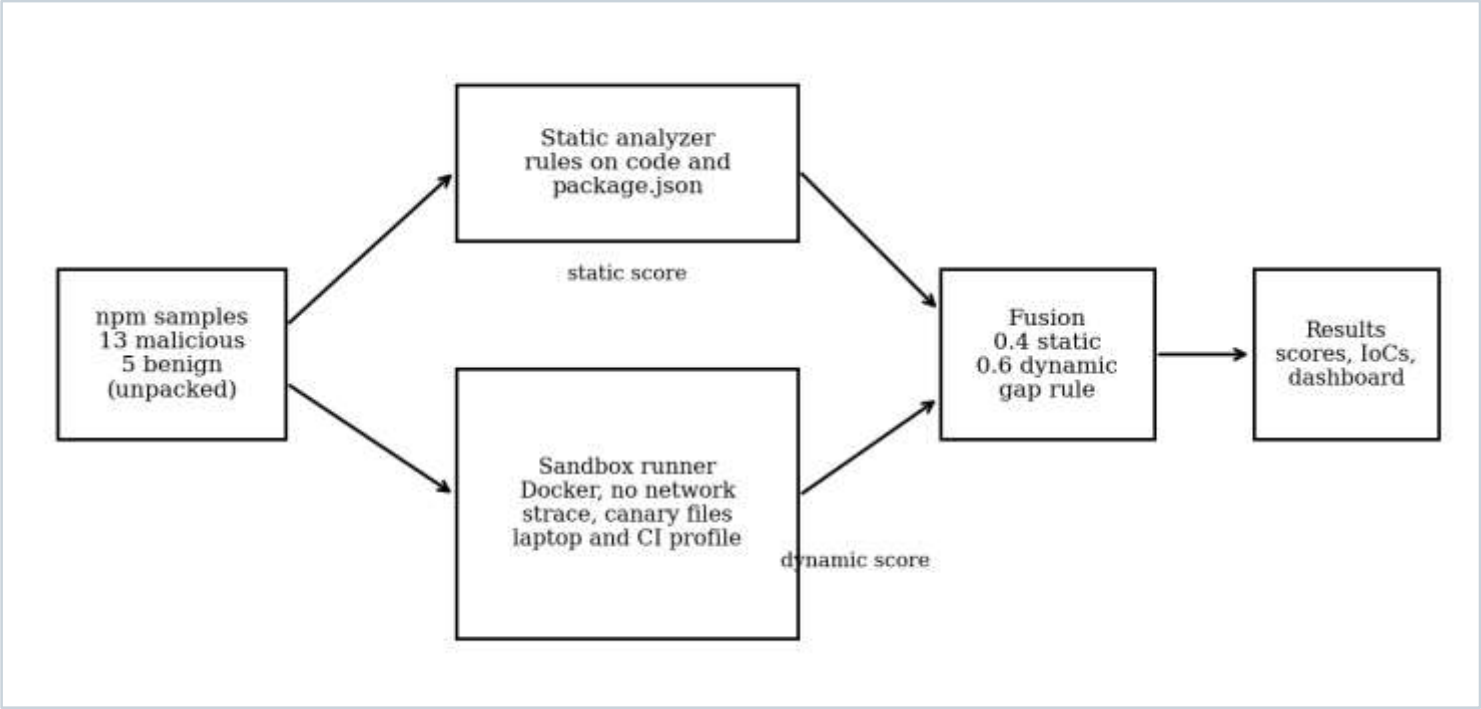


Figure 1: samples go to two analyzers, then fusion, then results

Output files: static_malicious.json, static_benign.json, dynamic_malicious.json, fusion_results.csv, iocs.csv, dashboard.html

Inside one sandbox run

- New container, network removed
- 512 MB memory, 256 processes, 1 CPU
- Four fake credential files planted: .npmrc, SSH key, AWS, git
- Phase 1: preinstall, install, postinstall, 20 s each
- Phase 2: import for 12 s with timers shortened
- strace logs opens, programs, sockets
- Repeat as CI profile; ENV_DIFF if behaviour differs

Tools installed

Tool	Version or detail
Kali Linux	VM in VirtualBox, host-only adapter
Docker	Image npmshield-sandbox from node:20-slim
Node.js	v20.20.2 inside the image
strace	6.1 inside the image
tcpdump	4.99.3 inside the image
Python 3	All analysis scripts

Internet was used once, for strace and tcpdump. Adapter went back to host-only and the snapshot sandbox-ready was taken.



Figure 2: image built in about 15 s, tool versions printed

Dataset and malware samples collected

13 malicious packages

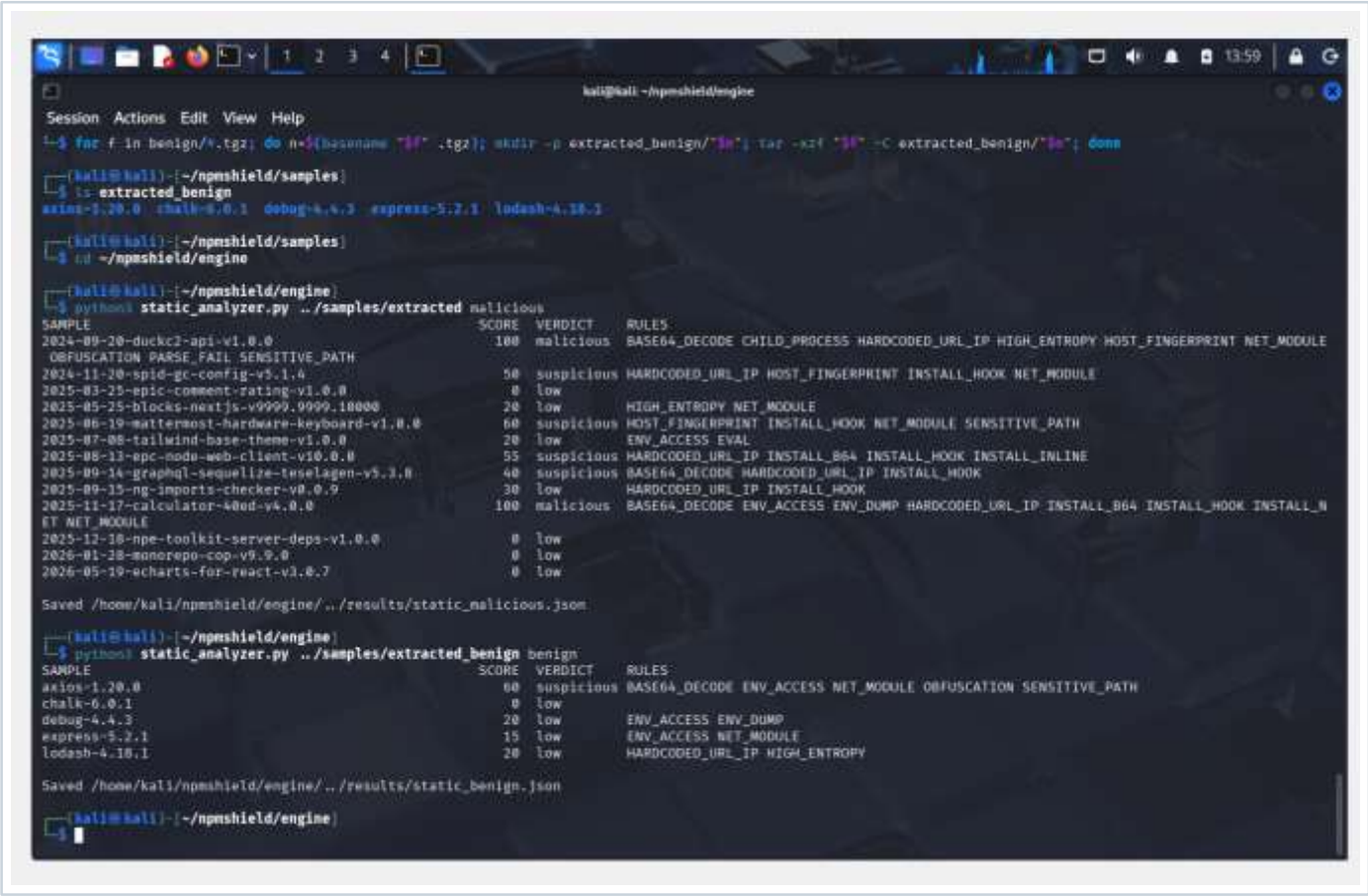
- duckc2-api 1.0.0
- epic-comment-rating 1.0.0
- mattermost-hardware-keyboard 1.0.0
- epc-node-web-client 10.0.0
- ng-imports-checker 0.0.9
- npe-toolkit-server-deps 1.0.0
- echarts-for-react 3.0.7
- spid-gc-config 5.1.4
- blocks-nextjs 9999.9999.10000
- tailwind-base-theme 1.0.0
- graphql-sequelize-teselagen 5.3.8
- calculator-40ed 4.0.0
- monorepo-cop 9.9.0

5 benign packages

- axios 1.20.0
- chalk 6.0.1
- debug 4.4.3
- express 5.2.1
- lodash 4.18.1

Published Sep 2024 to May 2026. Labelled malicious_intent in the source dataset. Real packages only.

Static analysis results



6 of 13

malicious packages flagged statically

1 of 5

false positive: axios scored 60

4 inert

scored 0 and checked by hand: placeholders, not misses

Figure 4: static_analyzer.py on the malicious and benign sets

Dynamic analysis results

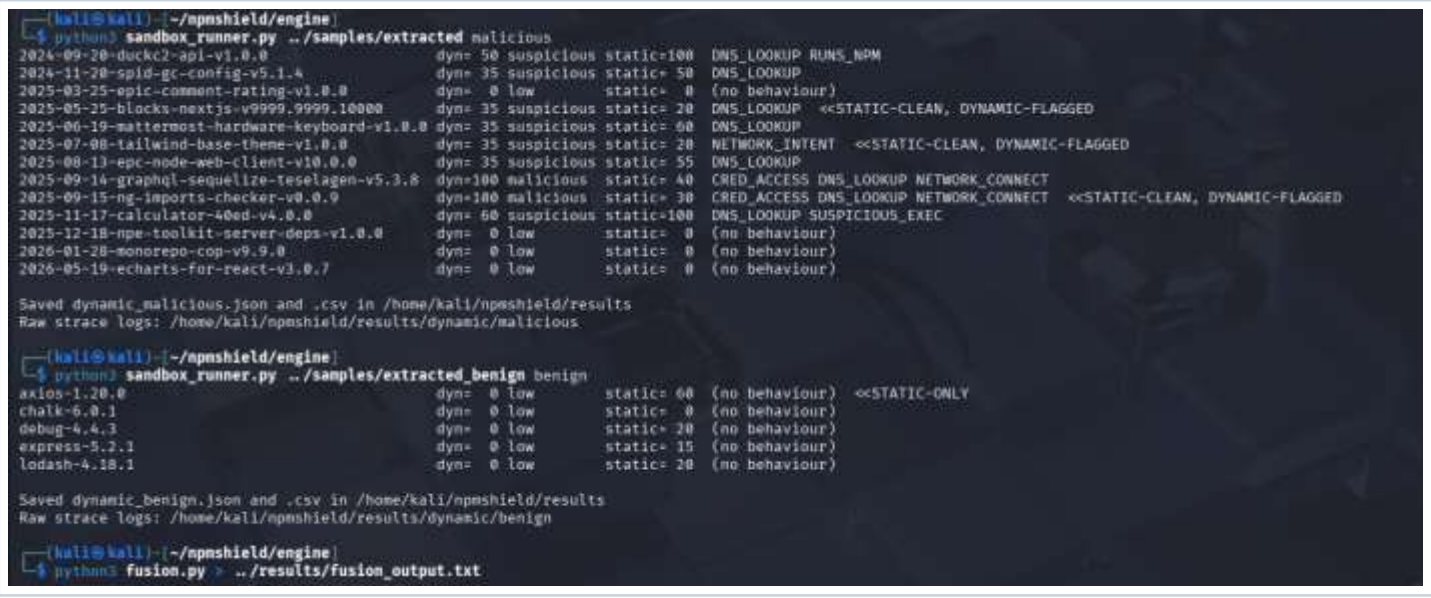


Figure 6: sandbox_runner.py on the malicious and benign sets

Flag	Points
NETWORK_CONNECT	40
DNS_LOOKUP	35
CRED_ACCESS	35
SUSPICIOUS_EXEC, DROPPED_EXEC	25
ENV_DIFF, RUNS_NPM	15

9 of 13

malicious packages flagged

0 of 5

benign packages flagged

0

ENV_DIFF hits between laptop and CI

What the sandbox recorded for each active package

Package	Recorded in the sandbox
ng-imports-checker, graphql-sequelize-teselagen	Read .npmrc and .aws/credentials; looked up registry.npmjs.org, api.github.com; tried 169.254.169.254:80
calculator-40ed	base64 of /etc/os-release, curl to an oastify.com host with the result in the path, second oastify.com lookup
tailwind-base-theme	axios.post to an ip-check-server address on vercel.app with environment variables in the body
blocks-nextjs	On import, looked up a unique oastify.com hostname
mattermost-hardware-keyboard	At install, looked up a unique oastify.com hostname
epc-node-web-client	At install, looked up wvzo30h2.h1.ci, also decoded from its preinstall base64
spid-gc-config	Looked up ipinfo.io and discord.com; code holds a hardcoded Discord webhook
duckc2-api	On import, ran npm install for networking packages; looked up raw.githubusercontent.com

Screenshots: fusion output

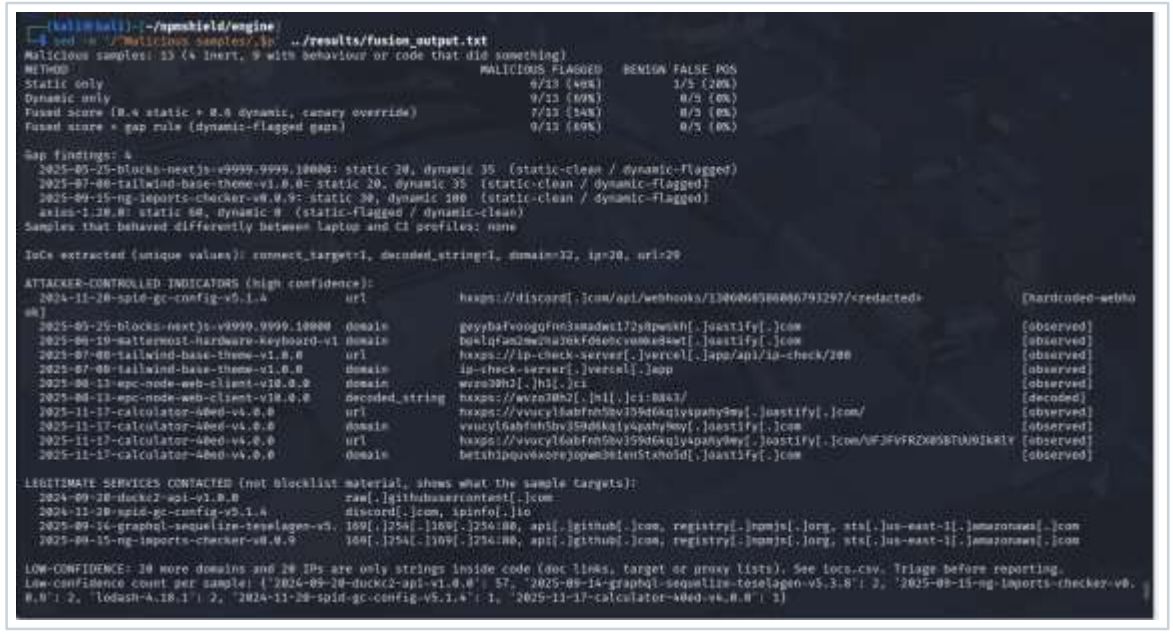
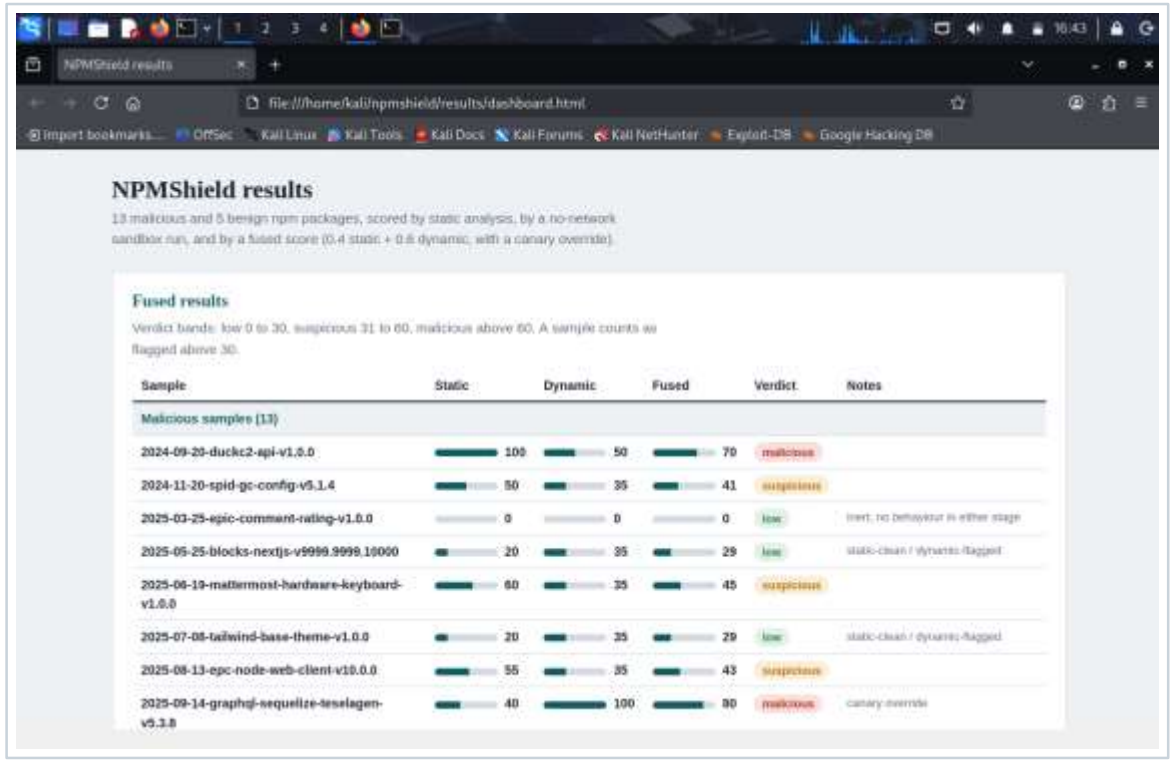


Figure 7: output of fusion.py



Figure 8: dynamic evidence printed by fusion.py

Screenshots: dashboard results



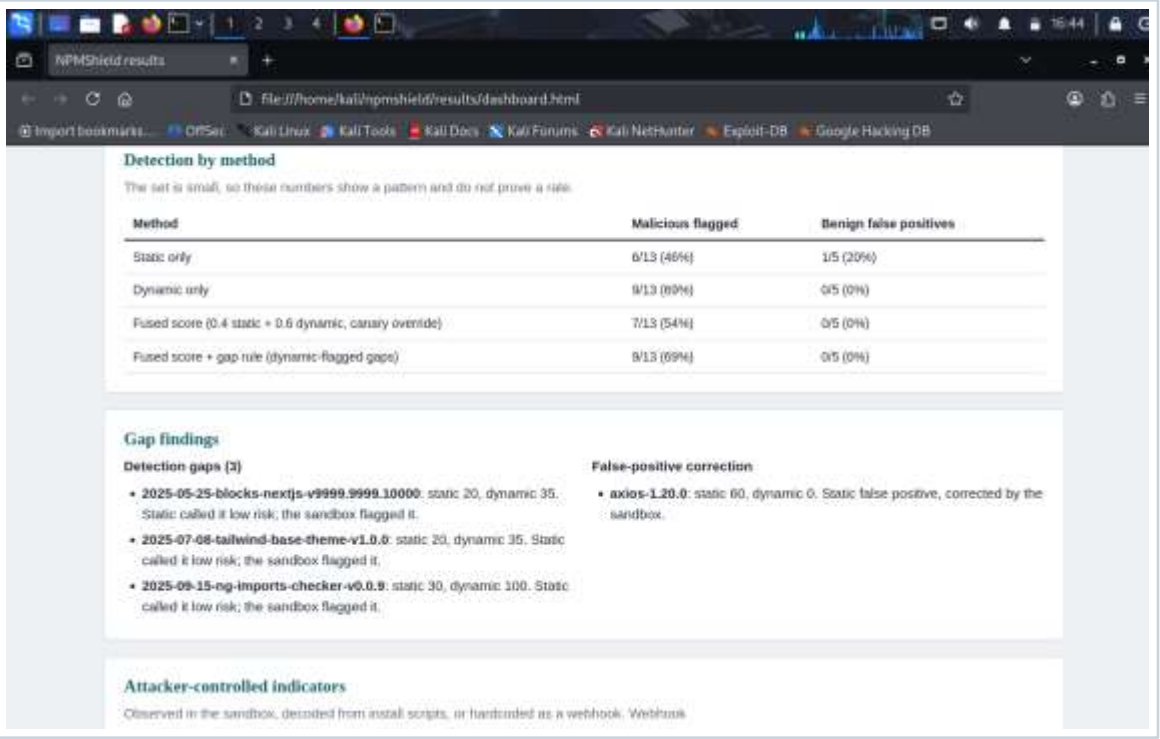


Figure 9: fused results table

Figure 10: detection by method and gap findings

Screenshots: indicators and sandbox evidence

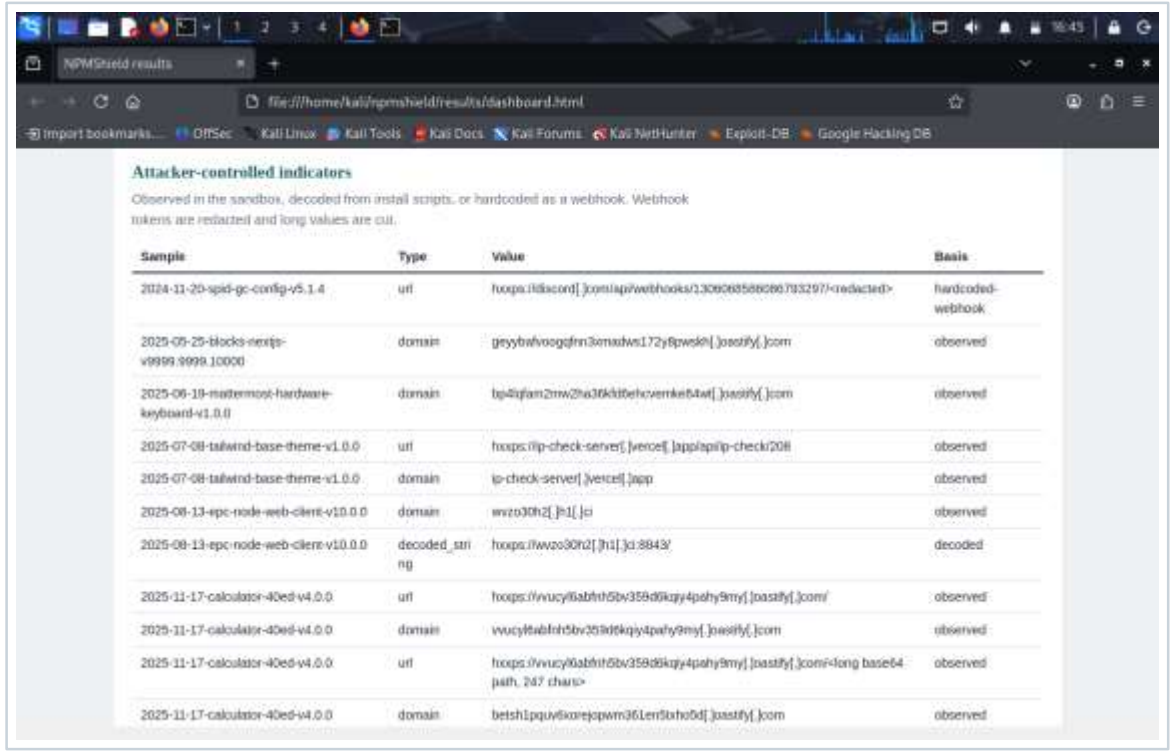


Figure 11: attacker-controlled indicators

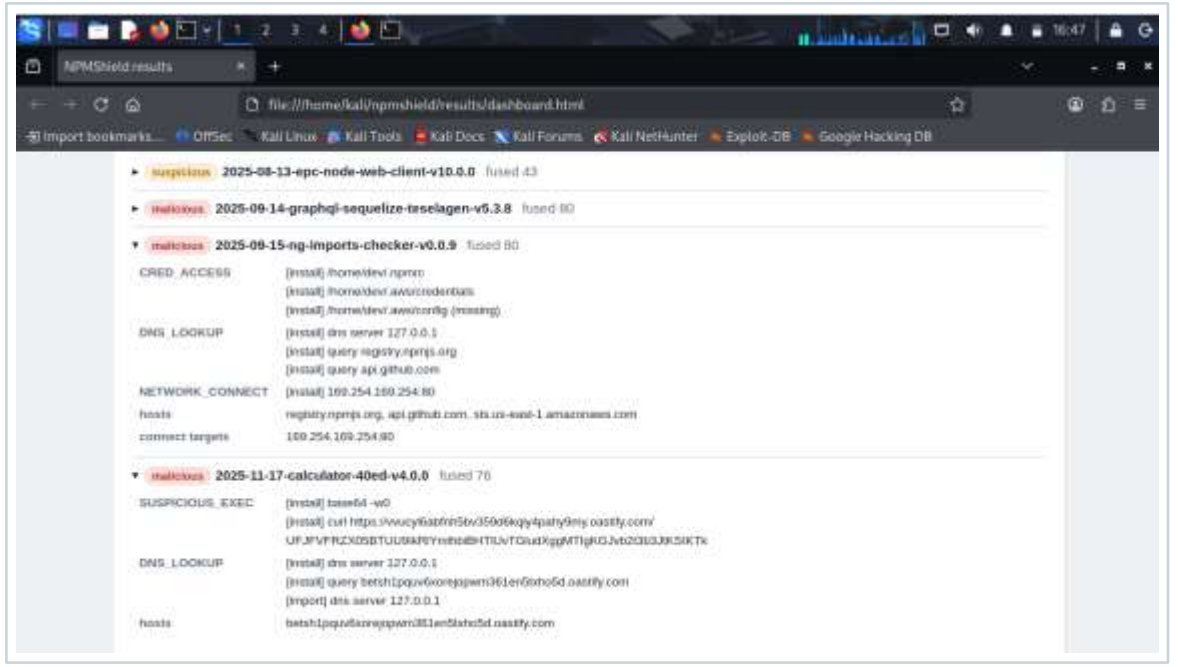
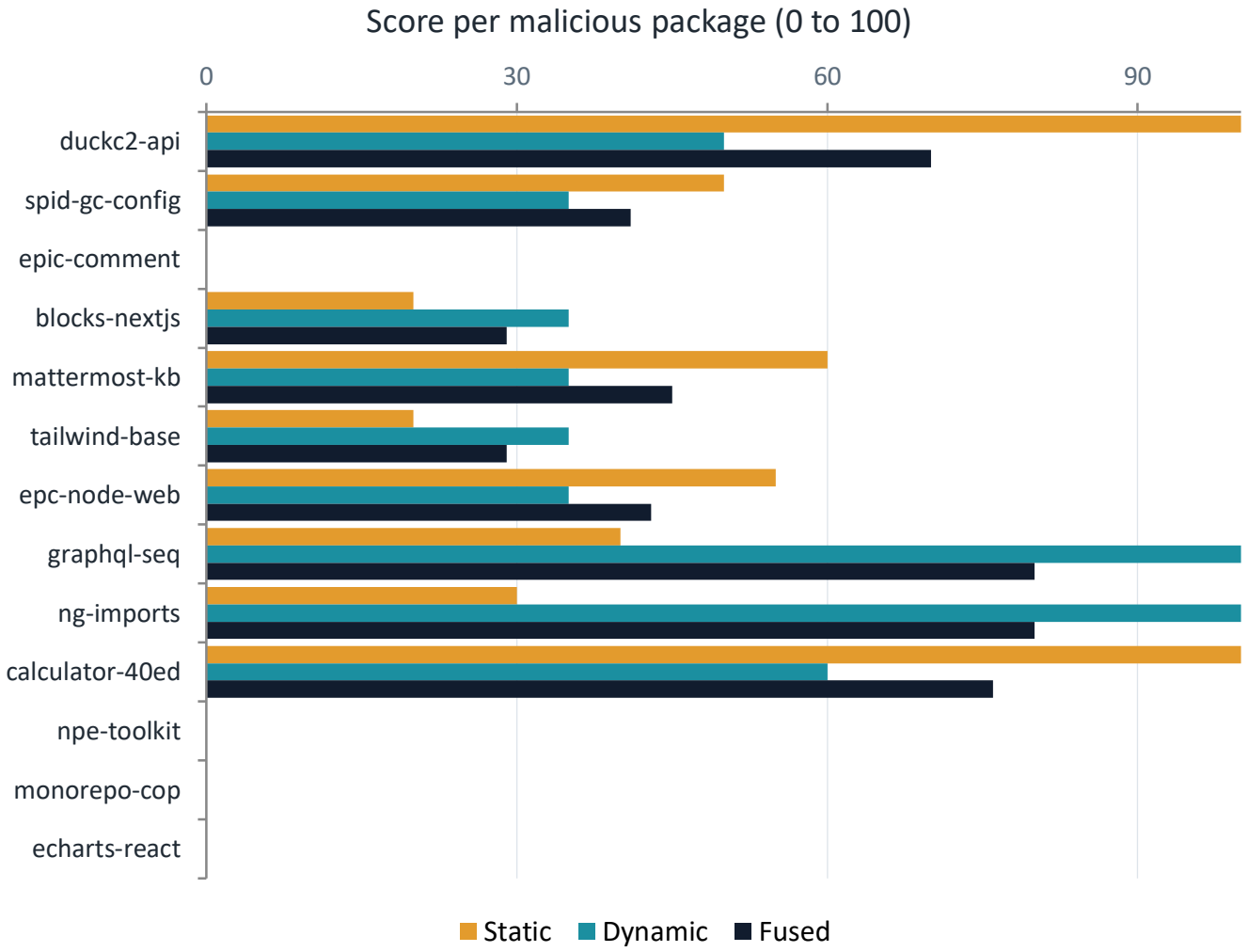


Figure 12: sandbox evidence per sample

Detection results: static, dynamic and fused



Method	Malicious flagged	Benign FP
Static only	6/13 (46%)	1/5
Dynamic only	9/13 (69%)	0/5
Fused score	7/13 (54%)	0/5
Fused + gap rule	**9/13 (69%)**	**0/5**

Accuracy on 14 samples (4 inert removed)

Static 71%, dynamic 100%, fused 86%, fused + gap 100%. Precision of fused + gap is 9/9.

Fusion matches dynamic on malware found and removes the axios false positive.

Important findings

1

Static-clean, sandbox-flagged

ng-imports-checker 30 vs 100, blocks-nextjs 20 vs 35, tailwind-base-theme 20 vs 35. Only running them exposed the behaviour.

2

Credential harvesting

Two packages opened the planted .npmrc and .aws/credentials at install and tried to reach 169.254.169.254.

3

Callback beacons

calculator-40ed and others call oastify.com hosts. That is Burp Suite out-of-band testing, so some may be research.

4

False positive corrected

axios scored 60 statically. The sandbox saw nothing, and the fused score is 24.

The tailwind-base-theme result comes through a stand-in for axios, because axios was not installed in the sandbox.

Problems encountered

Zero-score packages

Guessed a hidden dependency. Opened them: all inert placeholders. Count corrected.

No curl or wget in image

calculator-40ed failed with curl not found. Added recording stand-ins for both.

Missing modules and main file

Imports stopped early. Added logging stand-ins and a top-level import fallback.

Resolver noise

VirtualBox DNS 10.0.2.3 showed up as an indicator. Container resolver set to a local address.

npm mistaken for theft

duckc2-api ran npm install. Added RUNS_NPM and ignored npm's own file and DNS activity.

Noisy indicator list

32 domains and 20 IPs, mostly text. Split into three tiers. Discord token redacted.

Evidence for these is in Figures 13, 14 and 15 of the Review 2 report.

Work completed and work remaining

Completed

- ✓ Dataset collected and unpacked: 13 malicious, 5 benign
- ✓ Static analyzer built and run on all 18
- ✓ Docker sandbox image built; runner tested on all 18 with laptop and CI profiles
- ✓ Fusion score, gap rule and indicator extraction
- ✓ Results dashboard

Remaining

1. Find what duckc2-api fetches from raw.githubusercontent.com
2. Read spid-gc-config code to confirm what goes to the Discord webhook
3. Test date-triggered behaviour by moving the system clock
4. Install real dependencies so express and axios get a fair run
5. Final report and demonstration

Thank You